

Stage 2: Long-Horizon MimicGen Tasks

Why These Tasks, the Two-Simulator Problem, the Training Setup, and Current Status

Aryan Goyal

July 20, 2026 (updated July 23)

Abstract

Stage 1 ended with a precise open question. On short, contact-dominated tasks the predictor-driven chunk switch ties the best fixed chunk length but does not beat it. The theory says the win should appear on long tasks that alternate between free-space transport and contact-rich stages, because there no single fixed chunk length can be right everywhere. Stage 2 tests exactly that. It trains Diffusion Policy on four long-horizon MimicGen tasks (three_piece_assembly, nut_assembly, kitchen, coffee_preparation, all D0, 1,000 demonstrations each, mean episode lengths 337 to 689 steps) and will then run the same fixed- k versus predictor-switch comparison as Stage 1c. This document records the design decisions, the two-simulator version problem and how it was solved, the exact conversion and training pipeline, the completed trainings (all four tasks reached epoch 2,000), the first checkpoint-selection results (kitchen at or near ceiling, three_piece_assembly at 0.80, nut_assembly at 0.36, coffee_preparation still being evaluated), and the incident log: the July 21 scratch-disk event, the relaunch fault that deleted the surviving checkpoints, and the July 23 finding that both disk events were the machine being stopped, which reprovisions the scratch disk on this VM.

Contents

1	Why Stage 2 exists	3
2	The four tasks	3
3	The two-simulator problem	4
4	Step 1: dataset conversion	4
5	Step 2: policy training	5
6	Step 3: the evaluation bridge and checkpoint selection	5
6.1	Checkpoint selection results	6
7	Step 4, pending: the Stage 2 experiment	6
8	Status log, July 20 to 22, 2026	7
9	July 21: the disk event and the restart	8
9.1	July 21, afternoon: the relaunch destroyed the surviving checkpoints	8
9.2	July 22 to 23: trainings complete; the disk events explained	9

1 Why Stage 2 exists

Stage 1c’s pooled result is the starting point. On `tool_hang` the predictor switch reached 0.800 over three seeds, which statistically ties both the best fixed chunk length (0.807) and the best contact-oracle cell (0.807), while avoiding the 0.60 to 0.67 failure of per-step replanning in contact. That is a tie, not a win, and the Stage 1a analysis explains why a tie may be the best possible outcome on such tasks: `square` and `tool_hang` rollouts spend 65 to 88 percent of their steps in contact, so a single well-chosen fixed k is nearly optimal and an adaptive switch has almost nothing to modulate.

The hypothesis makes a sharper prediction for tasks where no single k can be right. A long task that alternates several times between free-space transport (stable, wants long chunks) and contact-rich manipulation (unstable, wants short chunks) forces every fixed k into a compromise: short k pays the per-step replanning cost through the long stable stretches, long k carries open-loop error through the contact stages. An adaptive switch pays neither cost. Stage 2 is the fair test of that prediction.

Two properties matter when picking the tasks:

1. **Length and structure.** Episodes should be several hundred steps with multiple distinct manipulation stages separated by transport, so the stable and unstable segments both occupy meaningful fractions of the episode.
2. **Headroom.** Base-policy success should sit well below 1.0, so an execution-rule improvement is measurable. Stage 1a’s `square` result showed what happens without headroom: everything lands in a flat band.

2 The four tasks

MimicGen (Mandlekar et al., 2023) provides 1,000-demonstration datasets for multi-stage robot-suites tasks, generated automatically from small sets of human source demonstrations. We use the D0 variant of four core tasks, which is the default object-initialization distribution. The datasets ship as state-only trajectories (simulator states plus actions), which is why the conversion step in Section 4 exists.

Task	Demos	Transitions	Mean length	Length range
<code>three_piece_assembly_d0</code>	1,000	336,695	337	308 to 375
<code>nut_assembly_d0</code>	1,000	358,907	359	293 to 435
<code>kitchen_d0</code>	1,000	616,751	617	585 to 654
<code>coffee_preparation_d0</code>	1,000	689,273	689	591 to 760

In plain terms: `three_piece_assembly` inserts two pieces onto a base in sequence; `nut_assembly` places both a square and a round nut onto their pegs; `kitchen` runs a multi-stage cooking sequence involving a stove, a pot, and bread; `coffee_preparation` loads a mug and prepares the coffee machine across several steps. Each task chains grasp, transport, and precise-placement stages, which is exactly the alternating structure Section 1 asks for. For scale, the Stage 1 tasks were one-stage or two-stage tasks with 200 demonstrations each; these are two to five times longer per episode, have five times the demonstrations, and published imitation-learning success rates on them sit well below ceiling.

3 The two-simulator problem

Stage 2 has one structural complication that shaped everything else, so it gets its own section.

MimicGen environments require robosuite 1.4 (they are registered by the `mimicgen` package, which pins the older simulator API). Our Stage 1 policy-training stack lives on the current robomimic main branch, which is the only branch with the Diffusion Policy implementation, and it runs against robosuite 1.5. The two requirements cannot coexist in one Python environment. Additionally, the robomimic v0.3 release that matches the MimicGen ecosystem has no Diffusion Policy at all (its template set stops at BC, BCQ, CQL, HBC, IQL, IRIS, TD3-BC).

The resolution is two virtual environments with a strict division of labor:

- **The `mg venv`** (robosuite 1.4.1, mujoco 2.3.2, robomimic v0.3, mimicgen): everything that must instantiate a MimicGen environment. Today that is dataset conversion; later it is policy evaluation.
- **The `lh venv`** (robosuite 1.5, robomimic main): everything that touches Diffusion Policy. Today that is training; it can never create a MimicGen environment.

The immediate casualty is in-training rollout evaluation. Stage 1 trainings rolled out the policy every 50 epochs to track success; Stage 2 trainings cannot, because the training process (`lh venv`) cannot build the environment. Training therefore runs with rollouts disabled, and success is measured post hoc from saved checkpoints through the `mg venv` (Section 6). This was validated up front with a one-epoch smoke test of the full training pipeline on a converted MimicGen dataset, which completed cleanly.

4 Step 1: dataset conversion

The MimicGen hdf5 files store simulator states, not observations. The robomimic v0.3 script `dataset_states_to_obs.py` replays each demonstration inside the environment and renders the observations the policy will train on. Per task we render:

- Two cameras, `agentview` and `robot0_eye_in_hand`, at 84 by 84 pixels. This matches the robomimic image recipe the Stage 1 policies used.
- The standard low-dimensional states (end-effector pose, gripper).
- `--exclude-next-obs`, which roughly halves the file size; behavior cloning never reads next-observations.
- `--done_mode 2`, the convention used by the released robomimic image datasets.

Two mechanical details worth recording. First, the script only works on MimicGen data if `import mimicgen` runs before the environment is created, because that import registers the environment classes; the script has no hook for this, so we invoke it through a small `runpy` wrapper that performs the import, sets `sys.argv`, and then executes the script as `__main__`. Second, rendering is headless EGL, pinned per GPU with `MUJOCO_GL=egl` and `MUJOCO_EGL_DEVICE_ID`; EGL processes do not appear in `nvidia-smi`'s compute-apps listing, so the reliable liveness check is output-file growth, not GPU process tables.

Each of the four conversions ran on its own GPU (0 through 3), writing `<task>_image.hdf5` next to the source file and logging to `<task>_img_gen.log`. Measured durations and output sizes:

Task	Conversion time	Output size
nut_assembly_d0	78 min	14.9 GB
kitchen_d0	92 min	25.7 GB
three_piece_assembly_d0	104 min	14.1 GB
coffee_preparation_d0	119 min	29.0 GB

Sizes track transition count times the uncompressed two-camera image payload (about 42 KB per step); the full set is 84 GB against 2.4 TB free on the scratch disk.

5 Step 2: policy training

Each GPU chain launches its Diffusion Policy training automatically the moment its conversion finishes, so no idle time sits between the steps. The training configuration is generated by `impl/configs/gen_mg_configs.py` from the `robomimic diffusion_policy.json` template, and it deliberately changes as little as possible from the Stage 1 Panda trainings:

- **Identical budget.** The template sets `epoch_every_n_steps` to 100, so 2,000 epochs is 200,000 gradient steps regardless of dataset size. This is the same budget the Panda policies got, which keeps later comparisons about the tasks, not the training length.
- **Same observation recipe.** RGB `agentview` plus `robot0_eye_in_hand` through a 76 by 76 `CropRandomizer`; low-dimensional end-effector position, quaternion, and gripper state.
- **Rollouts and validation disabled.** Rollouts for the reason in Section 3. Validation because the MimicGen hdf5s ship without train/valid mask groups, so the filter keys are set to none and all 1,000 demonstrations train.
- **Checkpoints every 50 epochs**, giving 40 checkpoints per task for the post-hoc evaluation to scan.
- **Dataset cached in RAM** (`hdf5_cache_mode: all`); the converted files fit comfortably in this machine’s memory.

The measured pace is much faster than the Stage 1 trainings, and the reason is instructive. The Stage 1 wall-clock (days per task) was dominated by in-training rollout evaluation; with rollouts disabled and the dataset cached in RAM, an epoch here is just 100 gradient steps at about 8 steps per second plus bookkeeping, or 30 to 45 wall seconds. At that pace 2,000 epochs is 17 to 24 hours per task, and since all four run concurrently, the whole training phase fits in a day. The venv constraint that forced rollouts off, in other words, also collapsed the schedule.

6 Step 3: the evaluation bridge and checkpoint selection

Measuring success requires rolling out lh-venv-trained checkpoints inside mg-venv environments, and no single process can do both. Of the two candidate designs (a cross-venv policy server, or backporting Diffusion Policy into the mg venv), the policy server was built and is the one in use: `bridge_eval.py` runs in the mg venv, owns the MimicGen environment, spawns `policy_bridge.py` in the lh venv as a subprocess, and streams observations to it one step at a time over a pipe. This is the same pattern Stage 1c used to run the predictor next to the rollout process. Environment metadata comes from the converted dataset rather than the checkpoint, so the mg-venv side never has to unpickle an lh-venv torch checkpoint.

6.1 Checkpoint selection results

Conditions: every cell is 25 rollout episodes at seed 0 through the bridge, one task per GPU. Observations are the two 84 by 84 cameras (agentview and eye in hand) plus proprioception, frame stack 2, matching training. Rollout horizons are fixed per task from the demonstration length maxima with margin: 560 steps for three_piece_assembly, 650 for nut_assembly, 980 for kitchen, 1,140 for coffee_preparation. Checkpoints are taken from the 2,000-epoch grids at epochs 600, 1,000, 1,400, 1,700, and 2,000. With 25 episodes the standard error is about 0.1, which is coarse but sufficient to pick a plateau checkpoint; the selected checkpoint then gets full 50-episode cells in the experiment itself. The selection rule is highest success rate, ties broken toward the later epoch. The July 22 machine stop (Section 9.2) interrupted the pass; five cells are rerunning and are marked pending.

	ep. 600	ep. 1000	ep. 1400	ep. 1700	ep. 2000	best so far
three_piece_assembly	0.48	0.52	0.64	0.80	0.68	0.80 at 1700
nut_assembly	0.12	0.00	0.32	0.32	0.36	0.36 at 2000
kitchen	0.96	1.00	1.00	1.00	pending	1.00
coffee_preparation	0.04	pending	pending	pending	pending	open

Reading these as base policies for the switch experiment: kitchen is at or near ceiling, three_piece_assembly is strong, and nut_assembly is weak but usable, with enough headroom in all three for chunk-length effects to show in either direction. A ceiling-level task like kitchen may compress differences between cells; that is acceptable in a four-task design and is itself informative if every cell saturates. coffee_preparation cannot be judged from epoch 600 alone; nut_assembly scored 0.12 at 600 and tripled by 2000, so the later coffee checkpoints must land before deciding whether the task has enough success to carry a comparison. If it stays near zero, the stage proceeds on three tasks, which the four-task design anticipated.

7 Step 4, pending: the Stage 2 experiment

Once each task has a selected checkpoint, the experiment repeats the Stage 1c protocol on the long-horizon tasks: fixed $k \in \{1, 4, 8, 16\}$ and the predictor-driven (16, 4) switch, same policy weights across all cells, 50 episodes per seed, horizons set per task from the demonstration lengths. The prediction being tested: on these tasks the adaptive switch beats every fixed k , because the alternating stable/unstable structure makes every fixed choice pay one of the two costs from Section 1.

One decision is still open and it is the main scientific choice left in Stage 2: what the predictor head is at evaluation time.

1. **Zero-shot.** Use the frozen Stage 1 head as-is. The DINOv2 ablation showed the head reads scene configuration, and the MimicGen scenes are robosuite scenes visually adjacent to the training tasks, so transfer is plausible. This is the cheap default and the more impressive result if it works.
2. **Relabel and fine-tune.** Run the FTLE labeler over the MimicGen demonstrations (through the mg venv) and train a head that has seen these tasks. This is days of labeling compute and requires adapting the labeler to the mg environments, but it separates “the switch idea works on long-horizon tasks” from “the head transfers zero-shot,” which are different claims.

The honest sequencing is to run zero-shot first, since it needs nothing new, and to decide on relabeling based on what the zero-shot head’s predictions look like on MimicGen frames (sane $\hat{\lambda}$ distributions and plausible switching would justify skipping the relabel; degenerate always-long or always-short output would force it).

That check has now run (July 20, 320 stamps per task sampled from 40 demonstrations each, agentview clips plus proprioception exactly as in training):

	mean $\hat{\lambda}$	p5 to p95	frac $> \delta$	corr with episode time
three_piece_assembly	0.022	−0.008 to 0.046	0.19	−0.04
nut_assembly	0.017	−0.016 to 0.050	0.22	0.00
kitchen	0.019	−0.009 to 0.038	0.11	0.16
coffee_preparation	0.035	0.002 to 0.058	0.55	0.26

The distributions are varied, span the training threshold $\delta = 0.0355$ on every task, and would drive real switching (11 to 55 percent of stamps land above δ). They are also ordered plausibly: coffee_preparation, the most contact-heavy manipulation sequence, reads most unstable, while kitchen, which is dominated by transport between fixtures, reads most stable. The decision is therefore zero-shot first. Two limits of this check are worth keeping in mind: it ran on demonstration frames, not on frames under policy control, and it verifies non-degeneracy, not accuracy; no ground-truth labels exist on these tasks, so the rollout comparison itself remains the real test. Relabeling stays available as the follow-up if the zero-shot switch underperforms.

8 Status log, July 20 to 22, 2026

- **09:05 UTC.** All four conversions launched, one per GPU, after the pipeline smoke test passed.
- **10:23 to 11:04 UTC.** Conversions completed in launch order of their durations (nut_assembly 10:23, kitchen 10:37, three_piece_assembly 10:49, coffee_preparation 11:05), and each Diffusion Policy training auto-launched within a second of its conversion finishing. No failures.
- **11:25 UTC.** All four trainings verified stepping: epoch progress at about 8 gradient steps per second on every GPU, roughly 4 GB of GPU memory each, 21 to 95 epochs completed, and the first periodic checkpoint (epoch 50) already on disk for nut_assembly.
- **Projection.** At the measured 30 to 45 seconds per epoch, the four trainings complete the morning of July 21 UTC, staggered in launch order.
- **12:15 UTC.** The evaluation bridge (Section 6) built and smoke-tested end to end on a partially trained nut_assembly checkpoint: full protocol round trip at about 7.7 environment steps per second, which prices a 50-episode cell at roughly 70 minutes on one GPU. Three cross-venv format mismatches were found and fixed in the process (stdout noise in the protocol stream, v0.3-processed images versus main-branch raw-image expectations, and the checkpoint’s frame stacking, which the client now reproduces).
- **12:50 UTC.** The zero-shot distribution check (Section 7) ran and passed; the head is not degenerate on MimicGen frames and the zero-shot-first decision is taken.
- **Remaining before the experiment.** Checkpoint selection once trainings finish (morning of July 21), then the fixed- k sweep plus predictor cells.

9 July 21: the disk event and the restart

Overnight between July 20 and 21 the scratch disk was largely wiped: the source and converted datasets, the three Python environments, the cloned repositories, and the extracted feature files were deleted. The four training processes died at 05:01 UTC, at epochs 1,721 (nut_assembly), 1,653 (kitchen), 1,637 (three_piece_assembly), and 1,596 (coffee_preparation) of 2,000. They had survived the deletion itself for several hours (their code and RAM-cached data were held as open file handles) and then went down without a Python traceback, consistent with the disk being reprovisioned under them.

What survived, and why the damage is small:

- **All checkpoints and results** live on persistent storage and were untouched. Checkpoints saved every 50 epochs, so the surviving grids end at epoch 1,700 / 1,650 / 1,600 / 1,550 per task. All four are far past the epoch 950 to 1,200 region where Stage 1’s best checkpoints lived, so these are almost certainly at-plateau policies.
- **The labels** (31 npz files, days of GPU compute) survived on scratch and were immediately copied to persistent storage, along with the lift and can rollout datasets. The square and tool_hang rollout datasets were lost; they are only needed for future feature work, not for Stage 2.
- **Code and documents** live on persistent storage; nothing was lost.

The rebuild (venvs, repositories, dataset re-downloads, image reconversion) hit two reproducible faults worth recording. First, the fresh mg venv resolved a torchvision wheel that did not match its CUDA torch build; the fix is to install torch and torchvision together from the same CUDA index. Second, the fresh robomimic v0.3 clone restored an unconditional `import mujoco_py` in its robosuite environment wrapper; the original clone had carried a local try/except patch that was wiped with it. The patch is now written into the setup script so the next rebuild applies it automatically.

The restart decision, made at 12:21 UTC: run both tracks in parallel rather than serialize them.

- **GPUs 0 to 3:** reconvert the datasets (about two hours), then fresh 2,000-epoch training runs, completing around midday July 22. These reruns are completion insurance; nothing on the critical path waits for them.
- **GPUs 4 to 7:** checkpoint selection over the surviving checkpoint grids (epochs 600, 1,000, 1,400, and the last saved checkpoint per task), starting per task as its reconversion lands and finishing the evening of July 21. The Stage 2 experiment launches on the selected checkpoints, so the experiment timeline moves by less than a day despite the incident.

9.1 July 21, afternoon: the relaunch destroyed the surviving checkpoints

The parallel-tracks plan failed in its first hour, through the relaunch script rather than the disk. robomimic’s training entry point prompts for confirmation when its experiment directory already exists, and answering yes makes it delete that directory, checkpoints included. The relaunch command piped an automatic yes into that prompt. The result: the surviving checkpoint grids for all four tasks (epochs up to 1,700 / 1,650 / 1,600 / 1,550) were deleted at relaunch, and the

checkpoint selection pass that then ran found no checkpoint files and completed vacuously. The loss is permanent, because the same-day document backup had deliberately excluded the training outputs for size.

The fresh 2,000-epoch reruns, launched 13:39 to 14:19 UTC on the same datasets and configs, went from completion insurance to the pipeline itself. Three process changes came out of the failure. First, the launch script now refuses to start a training when the experiment directory exists, instead of answering the overwrite prompt; a human has to delete the directory deliberately. Second, checkpoint selection was rewritten to wait for each training to report success and then copy that task’s full checkpoint grid to the OS disk before evaluating anything, so no future fault can destroy the only copy. Third, the vacuous selection marker was cleared so the automatic shutdown watchdog (armed at the user’s request) cannot fire on it.

Net cost of the second incident: about one day. As of 06:30 UTC July 22 the four fresh runs are healthy at epochs 1,425 to 1,643 of 2,000, on pace to finish between roughly 10:00 and 13:00 UTC. Selection (epochs 600, 1,000, 1,400, 1,700, 2,000 per task, 25 episodes each) starts per task as its training lands, and the switch-versus-fixed- k experiment follows on the selected checkpoints.

9.2 July 22 to 23: trainings complete; the disk events explained

All four trainings finished on July 22, each reaching epoch 2,000 and reporting success, and checkpoint selection completed 15 of its 20 cells (the results in Section 6.1). At 14:55 UTC the virtual machine was stopped externally, killing the kitchen epoch-2,000 evaluation mid-run and leaving the four coffee_preparation cells beyond epoch 600 unstarted. On restart the next morning the scratch disk was again empty, and the boot history closed the case on both incidents: this Azure VM’s scratch disk is reprovisioned on every machine stop. The July 21 “disk event” aligns with the end of the July 17 boot, and the July 22 loss aligns with the 14:55 stop. Nothing was failing at random; stopping the machine is what wipes scratch. The operational consequence is simple: the machine must stay up until the pipeline finishes, and power-off is left to the automatic shutdown watchdog, which fires only when trainings, selection, and the Stage 2 experiment have all reported completion.

The July 23 rebuild rebuilt the three venvs and re-downloaded the four datasets, and in doing so reproduced both faults recorded in Section 9 (the mismatched torchvision wheel, which silently killed all four image reconversions, and the unpatched mujoco_py import in the fresh robomimic v0.3 clone). Both fixes are now written into the setup scripts rather than applied by hand, and the reconversions plus the five pending selection cells are rerunning. Checkpoint grids were never at risk this time: all four live on persistent storage, with a second copy on the rebuilt scratch as overwrite insurance.

- **Zero-shot head accuracy is unverified.** The planned degeneracy check has run and passed (Section 7), so the switch will actually switch; whether it switches at the right moments is untested, because no ground-truth labels exist on these tasks. The distribution check also used demonstration frames, and frames under policy control may differ. Fallback if the switch underperforms: relabel and fine-tune.
- **Base-policy success is unknown until the bridge exists.** If a task trains to very low success, the switch comparison on it loses power. Four tasks were chosen partly so that one weak task does not sink the stage.
- **No in-training signal.** With rollouts and validation disabled, training health is only visible

through the loss curve until the bridge runs. The losses are being watched; divergence would be caught within a checkpoint interval.

- **Horizon choices for evaluation.** Rollout horizons must be long enough that slow-but-successful episodes are not truncated; they will be set from the demonstration length maxima with margin, per task.
- **Deferred relative to the original plan.** The original Stage 2 (ACT on ALOHA, execution-rule ablations) is deferred, not dropped; the long-horizon test was promoted because Stage 1c identified it as the decisive missing experiment. FurnitureSim remains parked on a manual download. LIBERO-Long policy training remains an open option for the idle GPUs.

10 Summary

Stage 1 showed the switch matches the best fixed chunk length on short contact-dominated tasks and explained why it cannot beat it there. Stage 2 asks the question the theory actually cares about, on four MimicGen tasks whose episodes are long, multi-stage, and alternate between transport and contact. The two-simulator conflict between MimicGen (robosuite 1.4, no Diffusion Policy) and our training stack (robosuite 1.5) is solved by a strict conversion-in-mg, training-in-lh split, at the cost of moving all evaluation post hoc. The first training runs were killed at 80 to 86 percent by the July 21 scratch-disk event (Section 9), the surviving checkpoint grids were then deleted by a fault in the relaunch script itself (Section 9.1), and the fresh full-length reruns carried the pipeline: all four finished at epoch 2,000 on July 22. Both disk events turned out to be the same thing, the machine being stopped, which reprovisions this VM’s scratch disk (Section 9.2); the machine now stays up until the pipeline’s own shutdown watchdog fires. Checkpoint selection is three quarters done and already usable: kitchen at or near ceiling, three_piece_assembly best 0.80 at epoch 1,700, nut_assembly best 0.36 at epoch 2,000, coffee_preparation pending its later checkpoints (Section 6.1). The remaining sequence is: the five rerunning selection cells, then the fixed- k sweep plus predictor cells on the selected checkpoints, then automatic shutdown.